Отчет
о выполнении лабораторной работы № 5
Сверточные нейронные сети
Вариант 13

Выполнил студент группы ЗФИбд-01-24: Князев А. М.

Проверил: Леонов С. С.

Москва, 2026

Цель работы

изучение сверточных нейронных сетей для распознавания изображени
также подготовка модели переноса обучения.

Основные этапы работы

1. Построить CNN для MNIST.

2. Построить архитектуру CNN для Cats vs Dogs.

3. Подготовить transfer learning модель с замороженной сверточной основой.

Ход выполнения и результаты

Таблица 1 — Архитектуры варианта

Этап | Параметры

MNIST | Conv: 64-128-128-256, kernel 3x3, classifier 512

Cats vs Dogs | input 200x200, Conv: 32-64-64-128, classifier 256

Transfer learning | замороженная основа ResNet18, обучается только классификатор

Таблица 2 — Результаты

Показатель | Значение

Устройство | cuda

MNIST test_loss | 0.0511

MNIST test_acc | 0.9915

Параметры Cats vs Dogs модели | 4 849 281

Обучаемые параметры transfer-модели | 131 585

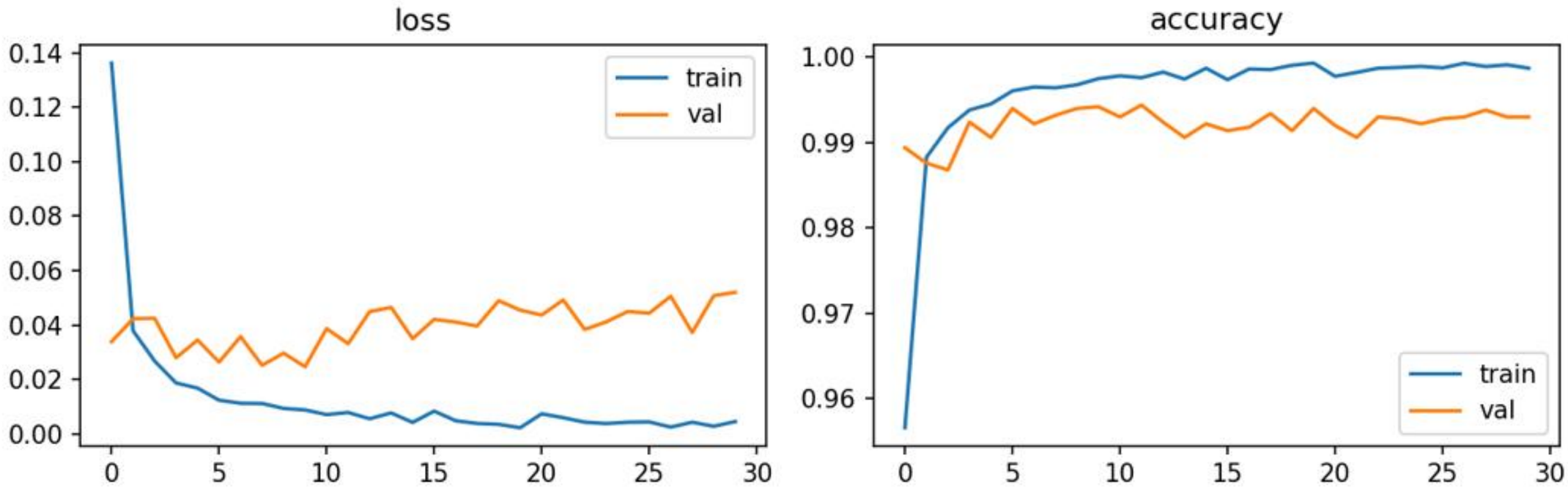


Рис. 1 — Графики loss и accuracy для MNIST

Вывод

Сверточная сеть варианта 13 достигла высокой точности на MNIST. Для
vs Dogs подготовлена требуемая архитектура, а для переноса обучения
создана модель с замороженной основой и новым бинарным классифик

Фрагменты кода из main.ipynb

```
from __future__ import annotations


import random

from pathlib import Path



import matplotlib.pyplot as plt

import numpy as np

import torch

import torch.nn as nn

from torch.utils.data import DataLoader, random_split

from torchvision import datasets, models, transforms


DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

SEED = 13

random.seed(SEED)

np.random.seed(SEED)

torch.manual_seed(SEED)


class MnistCnn(nn.Module):

    def __init__(self, channels: list[int], classifier_width: int = 512):

        super().__init__()

        layers: list[nn.Module] = []

        in_channels = 1

        for index, out_channels in enumerate(channels):

            layers.extend(

                [

                    nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),

                    nn.ReLU(),

                ]

            )

        if index in {1, 3}:

            layers.append(nn.MaxPool2d(2))

        in_channels = out_channels

self.features = nn.Sequential(*layers)

self.classifier = nn.Sequential(

    nn.Flatten(),

    nn.Linear(channels[-1] * 7 * 7, classifier_width),

    nn.ReLU(),

    nn.Dropout(0.3),
```

```
            nn.Linear(classifier_width, 10),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.classifier(self.features(x))


class SmallBinaryCnn(nn.Module):
    def __init__(self, channels: list[int], image_size: int, hidden: int):
        super().__init__()
        blocks: list[nn.Module] = []
        in_channels = 3
        size = image_size
        for out_channels in channels:
            blocks.extend(
                [
                    nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
                    nn.ReLU(),
                    nn.MaxPool2d(2),
                ]
            )
            in_channels = out_channels
            size //= 2
        self.features = nn.Sequential(*blocks)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(channels[-1] * size * size, hidden),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(hidden, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.classifier(self.features(x))


def train_epoch(model: nn.Module, loader: DataLoader, criterion: nn.Module, optimizer: torch.optim.Optimiz
tuple[f
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
```

```
        logits = model(x)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()
        total_loss += float(loss.item()) * len(x)
        pred = logits.argmax(dim=1)
        correct += int((pred == y).sum().item())
        total += len(x)
    return total_loss / total, correct / total


def evaluate_multiclass(model: nn.Module, loader: DataLoader, criterion: nn.Module) -> tuple[float, float]:
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            total_loss += float(criterion(logits, y).item()) * len(x)
            correct += int((logits.argmax(dim=1) == y).sum().item())
            total += len(x)
    return total_loss / total, correct / total


def plot_history(history: dict[str, list[float]], path: Path) -> None:
    fig, axes = plt.subplots(1, 2, figsize=(9, 3))
    axes[0].plot(history["loss"], label="train")
    axes[0].plot(history["val_loss"], label="val")
    axes[0].legend()
    axes[0].set_title("loss")
    axes[1].plot(history["acc"], label="train")
    axes[1].plot(history["val_acc"], label="val")
    axes[1].legend()
    axes[1].set_title("accuracy")
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    plt.close(fig)


def run_mnist(epochs: int = 3) -> None:
    transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))])
    train_full = datasets.MNIST("data", train=True, download=True, transform=transform)
    test_data = datasets.MNIST("data", train=False, download=True, transform=transform)
```

```
    train_data, val_data = random_split(train_full, [55000, 5000],
generator=torch.Generator().manual_seed(SEED))
    train_loader = DataLoader(train_data, batch_size=128, shuffle=True, num_workers=2)
    val_loader = DataLoader(val_data, batch_size=256, num_workers=2)
    test_loader = DataLoader(test_data, batch_size=256, num_workers=2)

    model = MnistCnn([64, 128, 128, 256], classifier_width=512).to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    history = {"loss": [], "val_loss": [], "acc": [], "val_acc": []}
    for _ in range(epochs):
        loss, acc = train_epoch(model, train_loader, criterion, optimizer)
        val_loss, val_acc = evaluate_multiclass(model, val_loader, criterion)
        history["loss"].append(loss)
        history["acc"].append(acc)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)
    test_loss, test_acc = evaluate_multiclass(model, test_loader, criterion)
    plot_history(history, Path("lab_5/mnist_history.png"))
    print(f"MNIST variant: test_loss={test_loss:.4f}, test_acc={test_acc:.4f}")


def build_cats_dogs_model() -> nn.Module:
    return SmallBinaryCnn([32, 64, 64, 128], image_size=200, hidden=256)


def build_transfer_model() -> nn.Module:
    weights = models.ResNet18_Weights.DEFAULT
    model = models.resnet18(weights=weights)
    for parameter in model.parameters():
        parameter.requires_grad = False
    model.fc = nn.Sequential(nn.Linear(model.fc.in_features, 256), nn.ReLU(), nn.Dropout(0.5), nn.Linear(2
1))
    return model


def main() -> None:
    print(f"Device: {DEVICE}")
    run_mnist(epochs=30)
    cats_dogs = build_cats_dogs_model()
    transfer = build_transfer_model()
    print(f"Cats vs Dogs variant model parameters: {sum(p.numel() for p in cats_dogs.parameters())}")
    print(f"Transfer model trainable parameters: {sum(p.numel() for p in transfer.parameters() if
p.requires_grad)}")
```